Using IP addresses for interprocess communication in the cloud is not an option.

To overcome that issue, you might consider using DNS records, relying on a round-robin name resolution pointing to one of the IP addresses assigned to the replicas. Knowing the hostname, you can reach the backing service even if one of the IP addresses changes because the DNS server would be updated with the new ones. However, this approach is not the best fit for cloud environments because the topology changes too often. Some DNS implementations cache the results of name lookups even after they should have expired. Similarly, some applications cache DNS lookup responses for too long. Either way, there's a high chance of using a hostname/IP address resolution that is no longer valid.

Service discovery in cloud environments requires a different solution. First, we need to keep track of all the service instances running and store that information in a *service registry*. Whenever a new instance is created, an entry should be added to the registry. When it's shut down, it should be removed accordingly. The registry recognizes that multiple instances of the same application can be up and running. When an application needs to call a backing service, it performs a *lookup* in the registry to determine which IP address to contact. If multiple instances are available, a *load-balancing* strategy is applied to distribute the workload across them.

We distinguish between client-side and server-side service discovery, depending on where the problem is solved. Let's take a look at both options.

7.3.2 Client-side service discovery and load balancing